Symplichain Hackathon Submission

Name: Abbagani Karthikeya

Role: Software Engineering Intern Applicant

Overview:

This repository contains my submission for the Symplichain Software Engineering Intern Hackathon.

The challenge focuses on solving real-world engineering problems across:

- Distributed systems (rate limiting & fairness)

- Mobile architecture

- CI/CD pipeline design

- Production debugging

Approach:

I focused on:

- Practical and scalable solutions

- Using the existing tech stack (Django, Redis, Celery, AWS)

- Clear reasoning instead of over-engineering

Project Structure

Part 1: Shared Gateway Problem:

Designed a request pooling and throttling system to:

- Enforce API rate limit of 3 requests/sec

- Ensure fairness across multiple customers

- Handle retries and failures

Key concepts:

- Redis-based queue

- Celery workers with rate limiting

- Round-robin scheduling for fairness

Part 2: Mobile Architecture:

Designed a mobile-first architecture using:

- React Native (cross-platform)

- Django REST APIs

- JWT-based authentication

Focus:

- Low-friction user experience

- Fast interactions for logistics workflows

Part 3: CI/CD Pipeline:

Implemented GitHub Actions workflow to:

- Automate deployment on staging & production

- Run migrations and static collection

- Deploy to EC2 using SSH

Suggested improvements:

- Docker for containerization

- Terraform for infrastructure automation

Part 4: Debugging Strategy

Defined a structured debugging approach:

- Step-by-step validation from API → S3 → Celery → AI → DB

- Use of CloudWatch, logs, and monitoring tools

- Root cause isolation methodology

ARCHITECTURE:

Clients

↓

Django API

↓

Redis Queue (per customer)

↓

Scheduler (round robin)

↓

Celery Worker (rate limited)

↓

External API

The system integrates with a third-party API that enforces a strict rate limit of 3 requests per second.

However, during peak hours, the platform generates more than 20 requests per second across 25 customers.

This leads to:

API failures (null responses)

Unreliable system behavior

The goal is to design a system that:

Ensures rate limit compliance (≤ 3 req/sec)

Handles high incoming traffic

Maintains fairness across customers

Handles failures gracefully

Architecture Design

Components Used

I designed the system using components already present in the existing tech stack:

Django API → Receives incoming requests

Redis → Acts as a message queue and temporary storage

Celery Workers → Process queued requests asynchronously

Scheduler Layer → Ensures fair distribution of requests

External API → Third-party service with rate limit

Architecture Flow

Client sends request to Django API

Instead of directly calling external API, request is pushed to Redis queue

Separate queues are maintained per customer

A scheduler picks requests from queues in a fair manner

Celery worker processes requests at a controlled rate

Request is sent to external API

Response is returned and stored if needed

MOBILE:

The goal is to build a mobile version of the SymFlow platform that is:

Fast

Easy to use (low friction)

Optimized for logistics/customer workflows

Primary: Click-based UI

I propose a click-based interaction model as the primary interface.

Why?

Most logistics users (drivers, operators) need:

Quick actions

Minimal thinking

Faster than typing or speaking

Example:

Upload POD → single button

Track shipment → one tap

Secondary: AI/Natural Language (Optional)

Add a lightweight AI assistant for advanced queries

Examples:

“Show delayed shipments”

“Check delivery status for order 123”

This improves UX but should NOT replace main UI

Architecture Design

Mobile App (React Native)

↓

API Layer (Django/DRF)

↓

Backend Services

↓

Database (PostgreSQL)

↓

External Services (S3, AI APIs)

Key Features & Design Choices

1. Authentication

-JWT-based authentication

-Secure API communication

2. API Communication

-REST APIs using Django backend

-Use Axios / Fetch

3. State Management

-Redux / Context API

-Helps manage:

-User data

-Orders

-App state

Offline Support (Important for logistics)

-Since drivers may have poor network

-Cache important data locally

-Sync when network is available

Performance Optimization

-Lazy loading screens

-Image compression before upload (for POD)

-Background sync

3. YAML:

```yaml
name: CI/CD Pipeline
on:
  push:
    branches:
      - staging
      - main
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
    - name: Checkout Code
      uses: actions/checkout@v3
    - name: Setup Python
      uses: actions/setup-python@v4
      with:
        python-version: 3.10
    - name: Install Dependencies
      run: |
```

```
      pip install -r requirements.txt
    - name: Run Migrations
      run: |
        python manage.py migrate
    - name: Collect Static Files
      run: |
        python manage.py collectstatic --noinput
    - name: Deploy to EC2
      run: |
        echo "${{ secrets.SSH_KEY }}" > key.pem
        chmod 600 key.pem
        ssh -o StrictHostKeyChecking=no -i key.pem
ubuntu@YOUR_EC2_IP << 'EOF'
          cd /home/ubuntu/app
          git pull origin main
          source venv/bin/activate
          pip install -r requirements.txt
          python manage.py migrate
          python manage.py collectstatic --noinput
          sudo systemctl restart gunicorn
          sudo systemctl restart nginx
        EOF
```

-The pipeline is triggered on push to staging and main branches. It installs dependencies, runs migrations, collects static files, and deploys to EC2 using SSH.

Store in GitHub Secrets:

SSH private key

EC2 IP

DB credentials

1. Docker

I would containerize the application using Docker.

Why?

-Same environment everywhere

-Easier deployment

-No “works on my machine” issues

2. Terraform (Infra as Code)

I would use Terraform to automate AWS infrastructure.

Why?

-Reproducible infra

-Version-controlled

-No manual EC2 setup

3. Monitoring Improvements

Current: Basic CloudWatch

Improve:

-Add alerts

-Health checks

-Logging system

4. Rollback Strategy

If deployment fails:

-Revert to previous version

-Restart services

4. DEBUGGING:

I follow a top-down debugging approach, starting from the entry point (client request) and validating each component in the pipeline sequentially to isolate the failure.

Step 1: Check Django API (Entry Point)

Goal:Check if request is reaching backend

What to check:

-Any errors?

-Status codes?

-Request logs?

Tools:

-AWS CloudWatch logs

-EC2 instance logs

Example: sudo journalctl -u gunicorn -f

--If request NOT reaching → problem is in mobile app/network

Step 2: Check File Upload to S3

Goal:Verify if image is stored

What to check:

-Is file present in S3 bucket?

-Any permission errors?

Tools:

-AWS S3 Console

-Django logs

If NOT uploaded:

-Issue with:

-S3 credentials

-Bucket policy

-File handling code

Step 3: Check Celery Task Trigger

Goal:Verify background job execution

What to check:

-Is task triggered?

-Is it failing?

Tools:

-Celery Flower dashboard

-Worker logs

Example:

 # celery -A app worker --loglevel=info

If task not triggered:

-Issue in Django → Celery connection

Step 4: Check AI Model (Bedrock)

Goal:Verify model processing

What to check:

-API response errors

-Timeout issues

Tools:

-CloudWatch logs

-Application logs

Possible issues:

-Invalid input format

-Model timeout

-API failure

Step 5: Check Database (PostgreSQL)

Goal: Verify result storage

What to check:

-Is data written to DB?

-Any DB errors?

Tools:

-RDS Console

-SQL queries

If not saved:

-DB connection issue

-Transaction failure